

7

Moving Beyond Linearity

Cho đến đây, cuốn sách chủ yếu tập trung vào linear models. Linear models tương đối đơn giản để mô tả và triển khai, đồng thời có ưu thế về interpretation và inference. Tuy nhiên, standard linear regression có thể bị hạn chế đáng kể về predictive power, bởi linearity assumption hầu như luôn chỉ là một approximation và đôi khi là approximation kém. Ở Chapter 6, ta thấy có thể cải thiện least squares bằng ridge regression, lasso, principal components regression và các technique khác. Trong setting đó, improvement đến từ việc giảm complexity của linear model và do đó giảm variance của estimates. Nhưng ta vẫn đang dùng một linear model, nên mức cải thiện cũng có giới hạn. Trong chapter này, ta nói lỏng linearity assumption nhưng vẫn cố duy trì interpretability nhiều nhất có thể. Ta sẽ xét các extension rất đơn giản của linear models như polynomial regression và step functions, cùng các approach tinh vi hơn như splines, local regression và generalized additive models.

- **Polynomial regression** mở rộng linear model bằng cách thêm các predictor tạo ra từ lũy thừa của original predictor. Chẳng hạn, cubic regression dùng ba variable X , X^2 , và X^3 làm predictors. Cách này cung cấp một phương pháp đơn giản để tạo non-linear fit.
- **Step functions** chia range của một variable thành K region riêng biệt để tạo một qualitative variable; kết quả là một piecewise constant function.
- **Regression splines** flexible hơn polynomials và step functions, và thực chất mở rộng cả hai. Range của X được chia thành K region; trong mỗi region ta fit một polynomial, nhưng các polynomial được constrain để nối trơn tại boundaries, hay *knots*. Nếu chia interval thành đủ nhiều region, fit có thể cực kỳ flexible.
- **Smoothing splines** tương tự regression splines nhưng xuất hiện từ một formulation khác: minimize residual sum of squares với một smoothness penalty.
- **Local regression** giống splines ở mục tiêu tạo flexible fit, nhưng

khác ở chỗ các region có thể overlap và chuyển tiếp rất smooth.

- **Generalized additive models** mở rộng các method trên để xử lý multiple predictors.

Trong Sections 7.1–7.6, ta trình bày nhiều approach để model relationship giữa response Y và một predictor X theo cách flexible. Ở Section 7.7, ta sẽ thấy những approach này có thể được tích hợp liền mạch để model response Y như một function của nhiều predictors $X_1, \dots, X_p$.

7.1 Polynomial Regression

Theo truyền thống, cách chuẩn để mở rộng linear regression sang setting mà relationship giữa predictors và response là non-linear là thay standard linear model

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

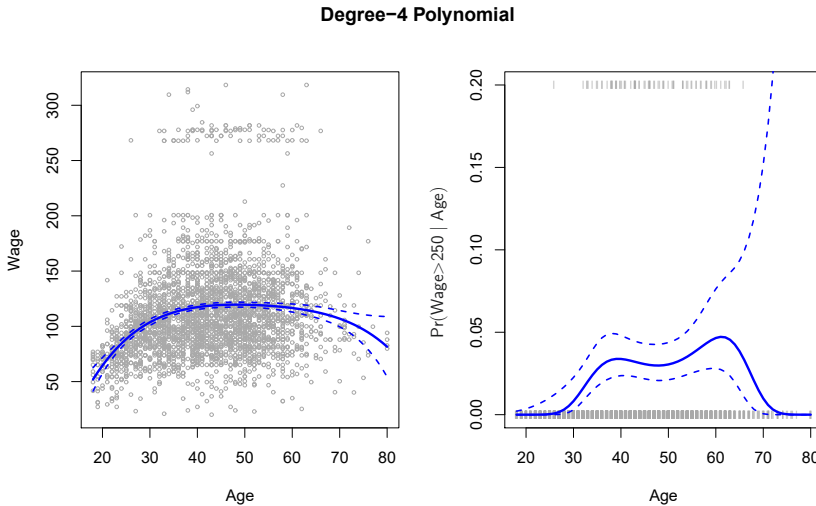
bằng một polynomial function

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \cdots + \beta_d x_i^d + \varepsilon_i. \quad (7.1)$$

Ở đây ε_i là error term. Approach này được gọi là *polynomial regression*; ta đã gặp một example ở Section 3.3.2. Với degree d đủ lớn, polynomial regression có thể tạo ra curve rất non-linear. Các coefficient trong (7.1) có thể estimate dễ dàng bằng least squares linear regression, vì đây chỉ là standard linear model với predictors $x_i, x_i^2, x_i^3, \dots, x_i^d$. Thông thường hiếm khi dùng d lớn hơn 3 hoặc 4, vì khi d lớn polynomial curve có thể quá flexible và mang shape rất kỳ lạ, đặc biệt gần boundary của variable X .

*polynomial
regression*

Left-hand panel của Figure 7.1 plot **wage** theo **age** cho dataset **Wage**, chứa information về income và demographic của nam giới sống ở central Atlantic region của Hoa Kỳ. Solid blue curve là degree-4 polynomial fit bằng least squares. Dù đây vẫn là linear regression model như các model khác, individual coefficients không phải trọng tâm; thay vào đó, ta xem toàn bộ fitted function trên grid 63 value của **age** từ 18 đến 80 để hiểu relationship giữa **age** và **wage**.



HÌNH 7.1. Dataset *Wage*. Left: Solid blue curve là degree-4 polynomial của *wage* (nghìn đô la) theo *age*, fit bằng least squares. Dashed curves là estimated 95% confidence interval. Right: Binary event $\text{wage} > 250$ được model bằng logistic regression, cũng với degree-4 polynomial. Fitted posterior probability rằng *wage* vượt \$250,000 được vẽ bằng blue curve cùng estimated 95% confidence interval.

Trong Figure 7.1, fit đi kèm một cặp dashed curve; đây là các curve ở mức $2 \times$ standard error. Giả sử ta đã tính fit tại một value cụ thể của *age*,

x_0 :

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4. \quad (7.2)$$

Ta muốn biết variance của fit, tức $\text{Var}\{\hat{f}(x_0)\}$. Least squares trả về variance estimate cho từng fitted coefficient $\hat{\beta}_j$, cùng covariance giữa từng pair coefficient estimate. Từ đó ta có thể tính estimated variance của $\hat{f}(x_0)$.¹ Estimated pointwise standard error của $\hat{f}(x_0)$ là square root của variance này. Ta lặp calculation tại từng reference point x_0 , rồi plot fitted curve cùng hai lần standard error ở hai phía. Với normally distributed error terms, khoảng ± 2 standard errors xấp xỉ 95% confidence interval.

Figure 7.1 gợi ý wages có thể đến từ hai population: high earners trên \$250,000 mỗi năm và low earners. Ta có thể biến **wage** thành binary variable bằng cách chia thành hai group này, rồi dùng logistic regression để predict binary response bằng polynomial functions của age. Cụ thể, ta fit

$$\Pr(y_i > 250 \mid x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}. \quad (7.3)$$

Kết quả nằm ở right-hand panel của Figure 7.1. Gray marks ở top và bottom cho biết ages của high earners và low earners. Solid blue curve biểu diễn fitted probability trở thành high earner theo age, cùng estimated 95% confidence interval. Confidence intervals khá rộng, nhất là phía bên phải. Mặc dù sample size của dataset lớn ($n = 3,000$), chỉ có 79 high earners, dẫn tới variance cao của estimated coefficients và do đó confidence intervals rộng.

7.2 Step Functions

Dùng polynomial functions của features làm predictors trong linear model áp đặt một global structure lên non-linear function của X . Ta có thể tránh global structure này bằng *step functions*: chia range của X thành các bin rồi fit một constant khác nhau trong mỗi bin. Điều này tương đương chuyển một continuous variable thành ordered categorical variable.

Cụ thể hơn, ta tạo cutpoints $c_1, c_2, \dots, c_K$ trong range của X , rồi tạo $K + 1$ variable mới

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \leq X < c_2), \\ C_2(X) &= I(c_2 \leq X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \leq X < c_K), \\ C_K(X) &= I(c_K \leq X). \end{aligned} \quad (7.4)$$

Ở đây $I(\cdot)$ là *indicator function*, trả về 1 nếu condition đúng và 0 nếu không. Chẳng hạn $I(c_K \leq X) = 1$ nếu $c_K \leq X$. Những variable này đôi khi gọi là dummy variables. Với mọi value của X , $C_0(X) + C_1(X) + \cdots + C_K(X) = 1$, vì X phải nằm đúng trong một trong $K + 1$ intervals.

¹Nếu $\tilde{C}$ là covariance matrix 5×5 của các $\hat{\beta}_j$, và $\ell_0^T = (1, x_0, x_0^2, x_0^3, x_0^4)$, thì $\text{Var}[\hat{f}(x_0)] = \ell_0^T \tilde{C} \ell_0$.

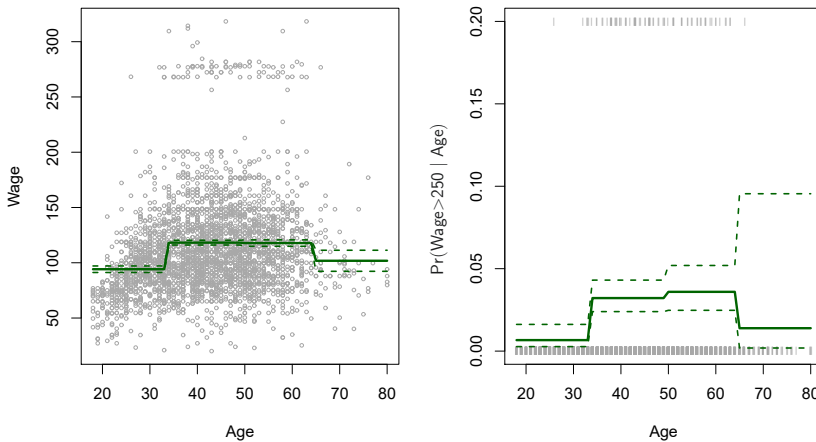
step function
ordered categorical
variable

indicator function

Sau đó ta dùng least squares để fit linear model với các predictors $C_1(X), \dots, C_K(X)$:

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \dots + \beta_K C_K(x_i) + \varepsilon_i. \quad (7.5)$$

² Với một value của X , nhiều nhất chỉ một trong $C_1, \dots, C_K$ khác zero. Khi $X < c_1$, toàn bộ predictors trong (7.5) đều zero, nên β_0 có thể được interpret là mean của Y cho $X < c_1$. Còn với $c_j \leq X < c_{j+1}$, model predict $\beta_0 + \beta_j$, nên β_j là average increase của response trong interval đó so với $X < c_1$.



HÌNH 7.2. Dataset *Wage*. Left: Solid curve là fitted value từ least squares regression của *wage* (nghìn đô la) dùng step functions của *age*; dashed curves là estimated 95% confidence interval. Right: Binary event $\text{wage} > 250$ được model bằng logistic regression dùng step functions của *age*; fitted posterior probability rằng *wage* vượt \$250,000 được hiển thị cùng estimated 95% confidence interval.

Left-hand panel Figure 7.2 minh họa việc fit step functions vào *Wage*. Ta cũng fit logistic regression model

$$\Pr(y_i > 250 \mid x_i) = \frac{\exp\{\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i)\}}{1 + \exp\{\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i)\}} \quad (7.6)$$

để predict probability một individual là high earner dựa trên age. Right-hand panel Figure 7.2 hiển thị fitted posterior probabilities từ approach này.

Nếu predictors không có natural breakpoints, piecewise-constant functions có thể bỏ lỡ structure. Chẳng hạn ở left-hand panel Figure 7.2, bin đầu tiên rõ ràng không capture increasing trend của wage theo age. Dù vậy, step function approach rất phổ biến trong biostatistics, epidemiology và nhiều discipline khác; ví dụ người ta thường dùng age group 5 năm để định nghĩa bins.

²Ta loại $C_0(X)$ khỏi predictors trong (7.5) vì nó redundant với intercept. Điều này tương tự việc chỉ cần hai dummy variable để code một qualitative variable có ba level khi model có intercept. Việc loại $C_0(X)$ thay vì một $C_k(X)$ khác là arbitrary. Hoặc có thể giữ $C_0(X), \dots, C_K(X)$ và bỏ intercept.

7.3 Basis Functions

Polynomial và piecewise-constant regression models thực ra là special cases của *basis function* approach. Ý tưởng là có sẵn một family function hoặc transformation áp dụng cho variable X : $b_1(X), b_2(X), \dots, b_K(X)$. Thay vì fit linear model trực tiếp theo X , ta fit

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \beta_3 b_3(x_i) + \dots + \beta_K b_K(x_i) + \varepsilon_i. \quad (7.7)$$

Các basis function $b_1(\cdot), \dots, b_K(\cdot)$ là fixed và known, tức được chọn trước. Với polynomial regression, $b_j(x_i) = x_i^j$; với piecewise constant function, $b_j(x_i) = I(c_j \leq x_i < c_{j+1})$. Ta có thể xem (7.7) như standard linear model với predictors $b_1(x_i), \dots, b_K(x_i)$, nên dùng least squares để estimate unknown regression coefficients. Quan trọng là toàn bộ inference tool cho linear models ở Chapter 3, như standard errors của coefficient estimates và F-statistics cho overall significance của model, vẫn dùng được.

Cho đến đây ta dùng polynomial functions và piecewise constant functions làm basis functions, nhưng còn nhiều lựa chọn khác, như wavelets hay Fourier series. Section tiếp theo xét một lựa chọn rất phổ biến: regression splines.

7.4 Regression Splines

Bây giờ ta xét một class basis function flexible hơn, mở rộng polynomial regression và piecewise constant regression.

7.4.1 Piecewise Polynomials

Thay vì fit một high-degree polynomial trên toàn range của X , *piecewise polynomial regression* fit các low-degree polynomial riêng trên những region khác nhau của X . Chẳng hạn, piecewise cubic polynomial dùng cubic regression

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \varepsilon_i, \quad (7.8)$$

nhưng coefficients $\beta_0, \beta_1, \beta_2, \beta_3$ khác nhau ở các phần khác nhau của range X . Các point nơi coefficients thay đổi được gọi là *knots*.

Ví dụ, piecewise cubic không có knot chỉ là standard cubic polynomial, tức (7.1) với $d = 3$. Với một knot tại c , model có dạng

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \varepsilon_i, & x_i < c, \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \varepsilon_i, & x_i \geq c. \end{cases}$$

Nói cách khác, ta fit hai polynomial function khác nhau: một cho observations có $x_i < c$, và một cho observations có $x_i \geq c$. Mỗi function có thể fit bằng least squares trên simple functions của original predictor.

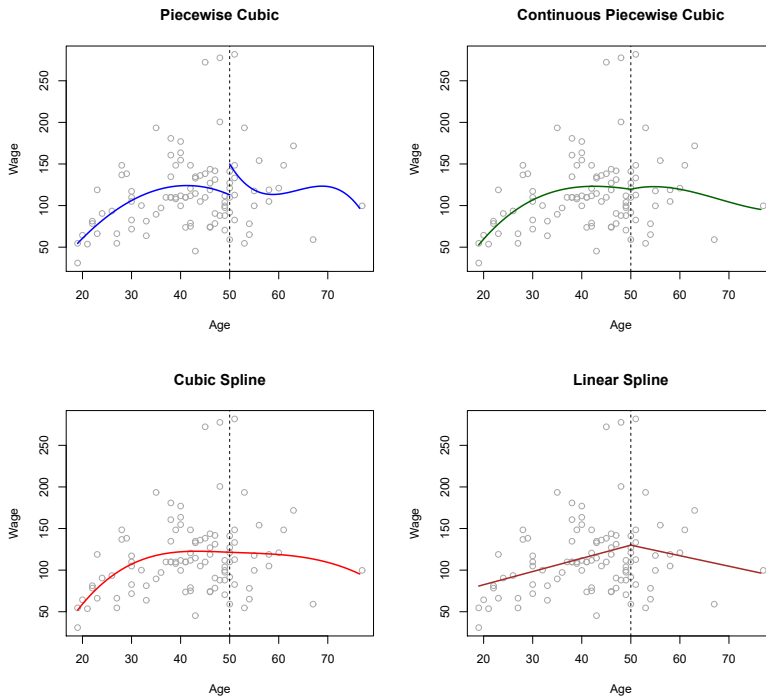
Dùng nhiều knot hơn tạo piecewise polynomial flexible hơn. Nếu đặt K knots trong range của X , ta sẽ fit $K + 1$ cubic polynomial khác nhau. Không bắt buộc phải dùng cubic: có thể dùng piecewise linear function, và piecewise constant functions ở Section 7.2 chính là degree-0 piecewise polynomials.

Top-left panel Figure 7.3 fit một piecewise cubic vào subset của *Wage*, với knot duy nhất tại $\text{age}=50$. Function bị discontinuous và trông không hợp

basis function

regression spline

*piecewise
polynomial
regression
knot*



HÌNH 7.3. Các piecewise polynomial khác nhau được fit vào một subset của *wage*, với một knot tại *age*=50. Top Left: Cubic polynomials không bị constrain. Top Right: Cubic polynomials được constrain continuous tại *age*=50. Bottom Left: Cubic polynomials được constrain continuous và có continuous first/second derivatives. Bottom Right: Linear spline được constrain continuous.

lý. Vì mỗi polynomial có bốn parameter, piecewise polynomial này dùng tổng cộng tám degrees of freedom.

degrees of freedom

7.4.2 Constraints and Splines

Top-left panel Figure 7.3 quá flexible. Để khắc phục, ta có thể fit piecewise polynomial dưới constraint rằng fitted curve phải continuous, tức không có jump tại age=50. Top-right panel cho fit này; nó tốt hơn nhưng V-shaped join vẫn không tự nhiên.

Ở lower-left panel, ta thêm hai constraint nữa: first và second derivatives của piecewise polynomials cũng continuous tại age=50. Như vậy function không chỉ continuous mà còn rất smooth. Mỗi constraint áp lên piecewise cubic polynomial thực chất giải phóng một degree of freedom bằng cách giảm complexity. Top-left dùng tám degrees of freedom; lower-left áp ba constraint (continuity, continuity của first derivative, continuity của second derivative), còn năm degrees of freedom. Curve này gọi là *cubic spline*. Nói chung, cubic spline với K knots dùng $4 + K$ degrees of freedom.³

cubic spline
derivative

Lower-right panel Figure 7.3 là *linear spline*, continuous tại age=50. Định nghĩa tổng quát: degree- d spline là piecewise degree- d polynomial, continuous ở các derivative đến degree $d - 1$ tại mỗi knot. Vì vậy linear spline được tạo bằng cách fit một line trong mỗi region do knots định nghĩa, với continuity tại từng knot. Ta có thể thêm nhiều knots hơn và impose continuity tại mỗi knot.

linear spline

7.4.3 The Spline Basis Representation

Regression splines ở section trước có vẻ phức tạp: làm sao fit piecewise degree- d polynomial dưới constraint rằng function (và có thể cả $d - 1$ derivative đầu) continuous? Ta có thể dùng basis model (7.7). Một cubic spline với K knots có thể được model như

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \varepsilon_i, \quad (7.9)$$

cho một lựa chọn thích hợp của basis functions $b_1, \dots, b_{K+3}$; model này fit bằng least squares.

Có nhiều representation tương đương của cubic spline. Cách trực tiếp nhất là bắt đầu với basis của cubic polynomial, x, x^2, x^3 , rồi thêm một *truncated power basis function* cho mỗi knot.

truncated power
basis

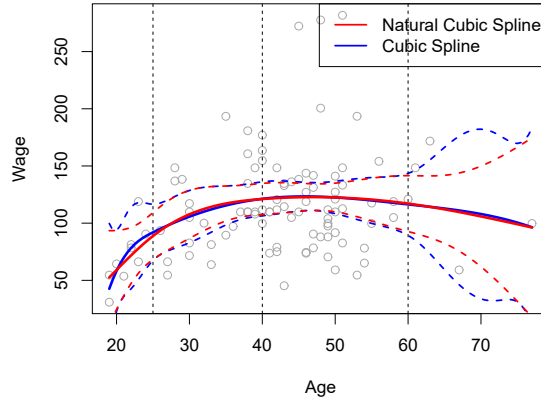
Truncated power basis function được định nghĩa

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi, \\ 0, & \text{otherwise.} \end{cases} \quad (7.10)$$

Ở đây ξ là knot. Có thể chứng minh rằng thêm term $\beta_4 h(x, \xi)$ vào cubic polynomial model (7.8) chỉ tạo discontinuity ở third derivative tại ξ ; function vẫn continuous và có continuous first/second derivatives tại knot.

Để fit cubic spline vào dataset có K knots, ta thực hiện least squares regression với intercept và $3 + K$ predictors: $X, X^2, X^3, h(X, \xi_1), \dots, h(X, \xi_K)$. Như vậy cần estimate tổng cộng $K + 4$ regression coefficients, tức cubic spline có $K + 4$ degrees of freedom.

³Cubic splines phổ biến vì mắt người thường không nhận ra discontinuity ở các knots.



HÌNH 7.4. Một cubic spline và một natural cubic spline với ba knots được fit vào subset của dataset *Wage*. Dashed lines đánh dấu knot locations.

Splines có thể có variance cao ở outer range của predictors, nơi X rất nhỏ hoặc rất lớn. Figure 7.4 cho fit trên *Wage* với ba knots; confidence bands ở boundary khá unstable. *Natural spline* là regression spline có thêm boundary constraints: function bắt buộc linear ở boundary region, tức nơi X nhỏ hơn smallest knot hoặc lớn hơn largest knot. Constraint này thường tạo estimates ổn định hơn ở boundaries. Figure 7.4 cũng hiển thị natural cubic spline bằng red line, với confidence intervals hẹp hơn.

natural spline

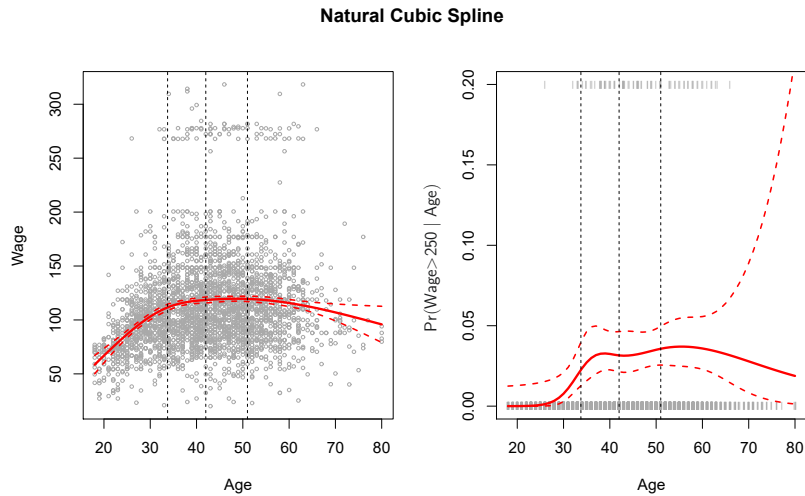
7.4.4 Choosing the Number and Locations of the Knots

Khi fit spline, nên đặt knots ở đâu? Regression spline flexible nhất ở region có nhiều knot, vì polynomial coefficients có thể thay đổi nhanh. Có thể đặt nhiều knots ở nơi ta tin function thay đổi nhanh và ít knots ở nơi ổn định hơn. Tuy nhiên trong practice, thường đặt knots một cách uniform; một cách là chỉ định desired degrees of freedom rồi để software tự đặt số knot tương ứng tại uniform quantiles của data.

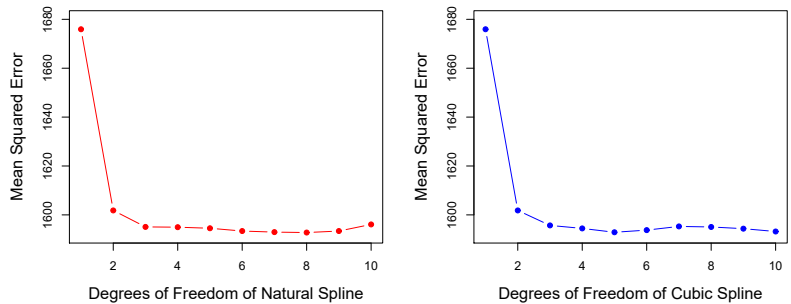
Figure 7.5 là example trên *Wage*. Tương tự Figure 7.4, ta fit natural cubic spline với ba knots, nhưng knot locations được chọn tự động là 25th, 50th và 75th percentiles của age. Điều này được request bằng bốn degrees of freedom. Lý do bốn degrees of freedom dẫn tới ba interior knots hơi technical.⁴

Ta nên dùng bao nhiêu knots, hay equivalently spline nên có bao nhiêu degrees of freedom? Có thể thử nhiều number of knots và nhìn curve, nhưng objective hơn là dùng cross-validation như Chapters 5 và 6. Ta bỏ ra một phần data (ví dụ 10%), fit spline với một number of knots nhất định trên phần còn lại, predict held-out portion, rồi lặp để mỗi observation được held out một lần và tính overall cross-validated RSS. Procedure được lặp cho các K khác nhau và chọn K có RSS nhỏ nhất.

⁴Thực tế có năm knots nếu tính cả hai boundary knots. Cubic spline với năm knots có chín degrees of freedom. Natural cubic spline có thêm hai natural constraints ở mỗi boundary để enforce linearity, nên còn $9 - 4 = 5$ degrees of freedom. Vì một constant được absorbed vào intercept, ta đếm là bốn degrees of freedom.



HÌNH 7.5. Một natural cubic spline với bốn degrees of freedom được fit vào dataset *Wage*. Left: Spline fit *wage* (nghìn đô la) theo *age*. Right: Logistic regression model binary event *wage* > 250 theo *age*; fitted posterior probability rằng *wage* vượt \$250,000 được hiển thị. Dashed lines là knot locations.



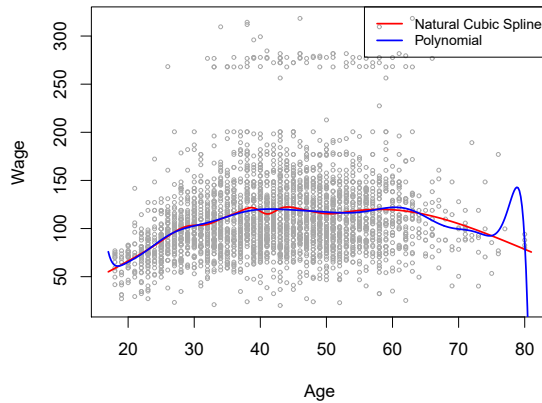
HÌNH 7.6. Ten-fold cross-validated mean squared errors để chọn degrees of freedom khi fit splines vào dataset *Wage*; response là *wage*, predictor là *age*. Left: Natural cubic spline. Right: Cubic spline.

Figure 7.6 cho ten-fold cross-validated MSE của splines với nhiều degrees of freedom. Natural cubic spline và cubic spline cho result gần như identical, với evidence rõ ràng one-degree fit (linear regression) là không đủ. Hai curve flatten rất nhanh; khoảng ba degrees of freedom cho natural spline và bốn cho cubic spline có vẻ đủ.

Ở Section 7.7 ta sẽ fit additive spline models đồng thời trên nhiều variable. Việc này có thể đòi chọn degrees of freedom cho từng variable; trong case như vậy, ta thường pragmatic hơn và đặt cùng một fixed number, chẳng hạn bốn, cho mọi term.

7.4.5 Comparison to Polynomial Regression

Figure 7.7 so sánh natural cubic spline với 15 degrees of freedom và degree-15 polynomial trên *Wage*. Extra flexibility của polynomial tạo undesirable behavior ở boundaries, trong khi natural cubic spline vẫn reasonable. Regression splines thường cho result tốt hơn polynomial regression vì polynomial muốn flexible phải tăng degree rất cao (ví dụ term X^{15}), còn splines tăng flexibility bằng cách tăng number of knots nhưng giữ degree cố định. Cách này thường cho estimate ổn định hơn. Splines còn cho phép đặt nhiều knots hơn ở region nơi f thay đổi nhanh và ít hơn ở nơi f ổn định.



HÌNH 7.7. Trên dataset *Wage*, natural cubic spline với 15 degrees of freedom được so sánh với degree-15 polynomial. Polynomials có thể có behavior rất mạnh, đặc biệt ở tails.

7.5 Smoothing Splines

Ở section trước ta tạo regression splines bằng cách chỉ định một set knots, sinh sequence basis functions, rồi dùng least squares để estimate spline coefficients. Bây giờ ta giới thiệu một approach khác cũng tạo ra spline.

7.5.1 An Overview of Smoothing Splines

Khi fit một smooth curve vào data, điều ta muốn là tìm function $g(x)$ fit observed data tốt, tức $\text{RSS} = \sum_{i=1}^n (y_i - g(x_i))^2$ nhỏ. Nhưng nếu không impose constraint nào lên g , ta luôn có thể làm RSS bằng zero bằng cách

chọn g interpolate toàn bộ y_i . Function như vậy sẽ overfit nặng vì quá flexible. Ta cần một g vừa làm RSS nhỏ vừa smooth.

Một cách tự nhiên là tìm function g minimize

$$\sum_{i=1}^n \{y_i - g(x_i)\}^2 + \lambda \int g''(t)^2 dt, \quad (7.11)$$

trong đó $\lambda \geq 0$ là tuning parameter. Function g minimize (7.11) được gọi là *smoothing spline*.

*smoothing spline
loss function*

Equation (7.11) có “Loss+Penalty” formulation giống ridge regression và lasso ở Chapter 6. Term $\sum_i (y_i - g(x_i))^2$ là loss function khuyến khích g fit data tốt, còn $\lambda \int g''(t)^2 dt$ là penalty chống variability của g . $g''(t)$ là second derivative; first derivative $g'(t)$ đo slope, còn second derivative đo mức slope thay đổi. Vì vậy $|g''(t)|$ lớn khi g rất wiggly quanh t và gần zero khi function smooth. Integral có thể hiểu như sum trên toàn range của t , nên $\int g''(t)^2 dt$ đo tổng roughness của g . Nếu g smooth, quantity này nhỏ; nếu g jumpy và variable, quantity này lớn. Do đó penalty khuyến khích smoothness, và λ càng lớn thì g càng smooth.

Khi $\lambda = 0$, penalty không có effect và g sẽ rất jumpy, interpolate chính xác training observations. Khi $\lambda \rightarrow \infty$, g hoàn toàn smooth: nó trở thành straight line fit training points tốt nhất, tức linear least squares line. Với intermediate λ , g approximate training observations nhưng vẫn smooth. Như vậy λ điều khiển bias-variance trade-off của smoothing spline.

Function $g(x)$ minimize (7.11) có special properties: nó là piecewise cubic polynomial với knots tại unique values $x_1, \dots, x_n$, có continuous first/second derivatives ở mỗi knot, và linear ngoài extreme knots. Nói cách khác, solution là natural cubic spline với knots tại $x_1, \dots, x_n$. Tuy nhiên nó không giống natural cubic spline nhận được nếu dùng basis-function approach ở Section 7.4.3 với knot tại mọi data point; đây là một shrunken version, trong đó λ điều khiển mức shrinkage.

7.5.2 Choosing the Smoothing Parameter λ

Smoothing spline là natural cubic spline có knot tại mọi unique x_i . Thoạt nhìn có vẻ nó sẽ có quá nhiều degrees of freedom, nhưng λ kiểm soát roughness và do đó *effective degrees of freedom*. Có thể chứng minh khi λ tăng từ 0 tới ∞ , effective degrees of freedom, ký hiệu df_λ , giảm từ n xuống 2.

*effective degrees of
freedom*

Tại sao lại nói effective degrees of freedom thay vì degrees of freedom? Thông thường degrees of freedom là số free parameters, như số coefficient trong polynomial hoặc cubic spline. Dù smoothing spline có n parameters và n nominal degrees of freedom, các parameter này bị constrain/shrink mạnh. Vì vậy df_λ là measure của flexibility: càng cao thì spline càng flexible, bias thấp hơn nhưng variance cao hơn.

Định nghĩa effective degrees of freedom hơi technical. Ta có thể viết

$$\hat{\mathbf{g}}_\lambda = S_\lambda \mathbf{y}, \quad (7.12)$$

trong đó $\hat{\mathbf{g}}_\lambda$ là solution của (7.11) với một λ cụ thể, tức n -vector chứa fitted

values tại training points $x_1, \dots, x_n$. Matrix S_λ là $n \times n$. Khi đó

$$\text{df}_\lambda = \sum_{i=1}^n \{S_\lambda\}_{ii}, \quad (7.13)$$

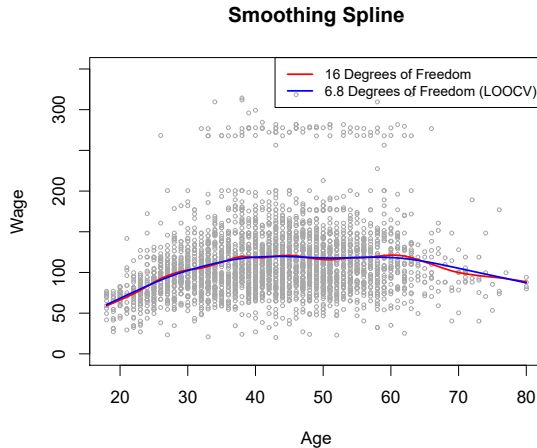
tức sum của diagonal elements của S_λ .

Khi fit smoothing spline, ta không cần chọn number/location of knots; knot đã nằm ở mọi training observation. Thay vào đó phải chọn λ . Cross-validation là một solution tự nhiên: tìm λ làm cross-validated RSS nhỏ nhất. Với smoothing splines, LOOCV error có thể compute rất efficiently, gần như cùng cost với một fit duy nhất:

$$\text{RSS}_{\text{CV}}(\lambda) = \sum_{i=1}^n \{y_i - \hat{g}_\lambda^{(-i)}(x_i)\}^2 = \sum_{i=1}^n \left\{ \frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{S_\lambda\}_{ii}} \right\}^2.$$

Notation $\hat{g}_\lambda^{(-i)}(x_i)$ là fitted value tại x_i khi fit dùng mọi training observation trừ observation thứ i ; còn $\hat{g}_\lambda(x_i)$ dùng toàn bộ training observations. Formula này cho phép tính từng leave-one-out fit chỉ từ original fit $\hat{g}_\lambda$.⁵ Ta đã gặp formula tương tự (5.2) cho least squares linear regression; nhờ đó LOOCV cũng có thể thực hiện nhanh cho regression splines và arbitrary basis functions fit bằng least squares.

Figure 7.8 cho smoothing-spline fits trên *Wage*. Red curve là fit khi pre-specify 16 effective degrees of freedom. Blue curve dùng λ chọn bằng LOOCV, tương ứng 6.8 effective degrees of freedom theo (7.13). Hai fit gần như không khác nhau, ngoài việc 16-df curve wiggly hơn một chút. Khi performance tương tự, fit 6.8 degrees of freedom preferable vì model đơn giản hơn nếu data không cung cấp evidence cho complexity thêm.



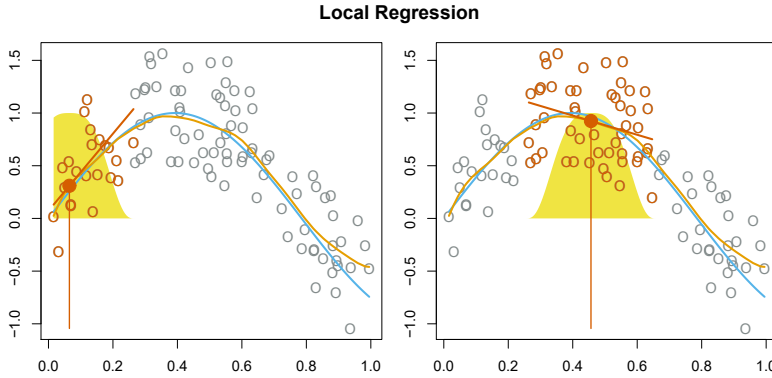
HÌNH 7.8. Smoothing-spline fits cho dataset *Wage*. Red curve dùng 16 effective degrees of freedom. Với blue curve, λ được chọn tự động bằng leave-one-out cross-validation, cho 6.8 effective degrees of freedom.

⁵Exact formulas để compute $\hat{g}(x_i)$ và S_λ khá technical; tuy nhiên có efficient algorithms để compute các quantity này.

7.6 Local Regression

Local regression là approach khác để fit flexible non-linear functions: tại target point x_0 , fit chỉ dùng nearby training observations. Figure 7.9 minh họa trên simulated data với một target point gần 0.4 và một point gần boundary 0.05. Blue line là true function $f(x)$ dùng để generate data; light orange line là local-regression estimate $\hat{f}(x)$. Procedure được tóm tắt trong Algorithm 7.1.

local regression



HÌNH 7.9. Local regression trên simulated data. Blue curve là $f(x)$ dùng để generate data, light orange curve là estimate $\hat{f}(x)$. Orange points là observations local quanh target x_0 (orange vertical line). Yellow bell-shape biểu diễn weights, giảm về zero khi distance tới target tăng. $\hat{f}(x_0)$ được obtain bằng weighted linear regression (orange line segment) rồi lấy fitted value tại x_0 (orange solid point).

Ở Step 3 của Algorithm 7.1, weights K_{i0} khác nhau cho mỗi x_0 . Do đó mỗi khi cần local-regression fit tại point mới, ta phải fit một weighted least squares regression mới bằng cách minimize (7.14) với set weights mới. Local regression đôi khi gọi là memory-based procedure vì, giống nearest-neighbors, ta cần toàn bộ training data mỗi lần compute prediction. Ta sẽ không đi sâu technical details ở đây.

Có nhiều choice khi perform local regression: weighting function K được define thế nào, và Step 3 fit linear, constant hay quadratic regression. Các choice đều có effect, nhưng quan trọng nhất là *span* s , proportion của points dùng để compute local regression tại x_0 . Span đóng vai trò tương tự λ trong smoothing splines: s nhỏ tạo fit local và wiggly hơn; s rất lớn tạo global fit dùng gần như mọi training observations. Có thể chọn s bằng cross-validation hoặc specify trực tiếp. Figure 7.10 dùng $s = 0.7$ và 0.2 trên Wage; đúng như kỳ vọng, $s = 0.7$ smooth hơn.

Local regression có thể generalize theo nhiều cách. Với multiple features $X_1, \dots, X_p$, một generalization hữu ích là fit multiple linear regression global theo một số variable nhưng local theo variable khác, chẳng hạn time. Những *varying coefficient models* như vậy thích hợp để adapt model theo data mới thu thập. Local regression cũng mở rộng tự nhiên sang setting local theo pair X_1, X_2 , bằng two-dimensional neighborhoods và bivariate linear regressions. Về lý thuyết có thể lên dimension cao hơn, nhưng lo-

cal regression thường perform kém khi p lớn hơn khoảng 3 hoặc 4 vì rất ít training observations ở gần x_0 . Nearest-neighbors regression cũng gặp problem tương tự ở high dimensions.

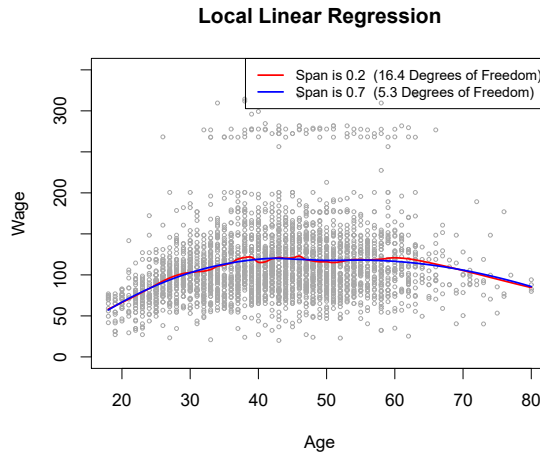
varying coefficient
model

Algorithm 7.1 Local Regression At $X = x_0$

1. Lấy fraction $s = k/n$ training points có x_i gần x_0 nhất.
2. Gán weight $K_{i0} = K(x_i, x_0)$ cho mỗi point trong neighborhood, sao cho point xa x_0 nhất có weight zero và point gần nhất có weight lớn nhất. Mọi point ngoài k nearest neighbors nhận weight zero.
3. Fit weighted least squares regression của y_i theo x_i bằng các weight trên, tìm $\hat{\beta}_0, \hat{\beta}_1$ minimize

$$\sum_{i=1}^n K_{i0} (y_i - \beta_0 - \beta_1 x_i)^2. \quad (7.14)$$

4. Fitted value tại x_0 là $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.



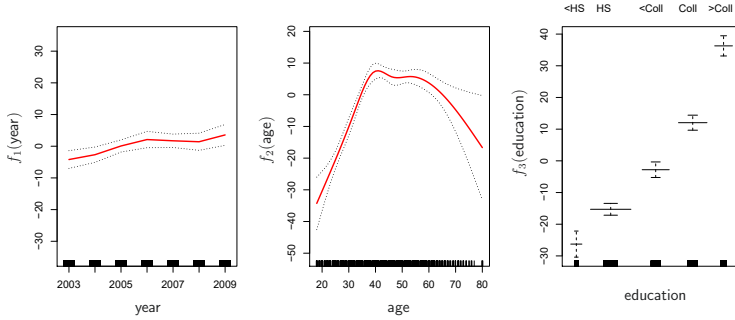
HÌNH 7.10. Local linear fits trên dataset *Wage*. Span chỉ fraction của data dùng để compute fit tại mỗi target point.

7.7 Generalized Additive Models

Sections 7.1–7.6 trình bày nhiều approach để flexibly predict response Y từ một predictor X , có thể xem là extensions của simple linear regression. Bây giờ ta xét việc flexibly predict Y từ nhiều predictors $X_1, \dots, X_p$, tức extension của multiple linear regression.

Generalized additive models (GAMs) cung cấp framework tổng quát để mở rộng standard linear model bằng cách cho phép non-linear function của từng variable nhưng vẫn giữ *additivity*. Giống linear models, GAMs dùng được với quantitative lẫn qualitative response. Trước hết ta xét quantitative response rồi đến qualitative response.

generalized additive
model
additivity



HÌNH 7.11. Với dataset *Wage*, relationship giữa từng feature và response *wage* trong fitted model (7.16). Mỗi panel hiển thị fitted function và pointwise standard errors. Hai function đầu là natural splines theo *year* và *age*, lần lượt 4 và 5 degrees of freedom. Function thứ ba là step function fit cho qualitative variable *education*.

7.7.1 GAMs for Regression Problems

Một cách tự nhiên để mở rộng multiple linear regression model

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

nhằm allow non-linear relationship giữa từng feature và response là thay mỗi linear component $\beta_j x_{ij}$ bằng một smooth non-linear function $f_j(x_{ij})$:

$$\begin{aligned} y_i &= \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \varepsilon_i \\ &= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i. \end{aligned} \quad (7.15)$$

Đây là một GAM. Nó gọi là additive model vì ta estimate riêng f_j cho từng X_j rồi cộng contributions lại.

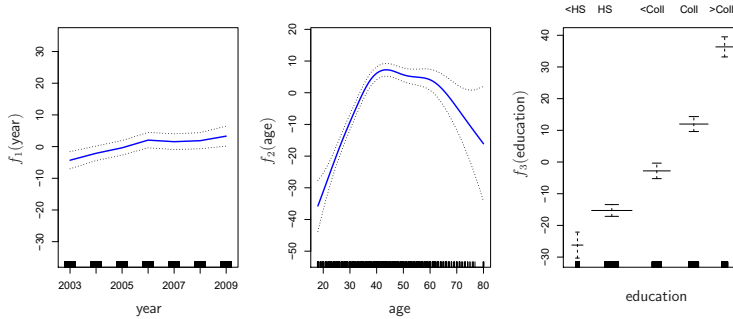
Các method fit single-variable function ở Sections 7.1–7.6 trở thành building blocks cho additive model. Ví dụ, xét model

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \varepsilon \quad (7.16)$$

trên *Wage*. *year* và *age* là quantitative, còn *education* là qualitative với năm level: <HS, HS, <Coll, Coll, >Coll. Ta fit hai function đầu bằng natural splines; function thứ ba bằng một constant riêng cho mỗi level, thông qua dummy-variable approach.

Figure 7.11 cho result của fitting (7.16) bằng least squares. Việc này đơn giản vì natural splines có thể construct bằng appropriate basis functions; toàn model chỉ là một regression lớn trên spline-basis variables và dummy variables trong một regression matrix.

Figure 7.11 dễ interpret. Left panel cho thấy khi giữ *age* và *education* fixed, *wage* tăng nhẹ theo *year*, có thể do inflation. Center panel cho thấy giữ *education* và *year* fixed, *wage* cao nhất ở intermediate ages và thấp ở rất trẻ/rất già. Right panel cho thấy giữ *year* và *age* fixed, *wage* tăng theo *education*; người học nhiều hơn có average salary cao hơn. Các finding đều intuitive.



HÌNH 7.12. Như Figure 7.11, nhưng f_1 và f_2 lần này là smoothing splines với lần lượt bốn và năm degrees of freedom.

Figure 7.12 cho ba plot tương tự, nhưng f_1 và f_2 là smoothing splines với lần lượt bốn và năm degrees of freedom. Fit GAM với smoothing spline không đơn giản như natural spline vì least squares không dùng trực tiếp được. Standard software như Python package `pygam` có thể fit GAM smoothing splines bằng approach gọi là *backfitting*: lặp lại việc update fit cho từng predictor trong khi giữ các predictor khác fixed. Mỗi lần update một function, ta chỉ apply fitting method cho variable đó lên một partial residual.⁶

*pygam
backfitting*

Fitted functions trong Figures 7.11 và 7.12 khá giống nhau. Trong đa số setting, difference giữa GAM dùng smoothing splines và natural splines là nhỏ.

Ta không bắt buộc dùng splines làm building blocks của GAMs: có thể dùng local regression, polynomial regression hoặc bất kỳ combination nào của các approach trước đó. GAMs sẽ được investigate thêm trong lab cuối chapter.

Pros and Cons of GAMs

Trước khi tiếp tục, ta tóm tắt advantage và limitation của GAM.

- GAMs cho phép fit non-linear f_j cho từng X_j , tự động model non-linear relationships mà standard linear regression bỏ lỡ; không cần thử thủ công nhiều transformation cho từng variable.
- Non-linear fits có thể tạo prediction chính xác hơn cho response Y .
- Vì model additive, ta có thể examine effect của từng X_j lên Y riêng rẽ trong khi giữ các variable khác fixed.
- Smoothness của f_j cho X_j có thể summarize bằng degrees of freedom.
- Limitation chính là GAM bị restrict additive, nên khi có nhiều variables có thể miss important interactions. Tuy vậy, như linear regression, có thể add interaction terms thủ công bằng predictors dạng $X_j \times X_k$, hoặc add low-dimensional interaction function $f_{jk}(X_j, X_k)$ fit bằng two-dimensional smoothers như local regression hay two-

⁶Ví dụ partial residual cho X_3 là $r_i = y_i - f_1(x_{i1}) - f_2(x_{i2})$. Nếu biết f_1, f_2 , ta fit f_3 bằng cách xem residual này như response trong non-linear regression theo X_3 .

dimensional splines.

Với fully general models, cần flexible approaches hơn như random forests và boosting ở Chapter 8. GAMs là compromise hữu ích giữa linear và fully nonparametric models.

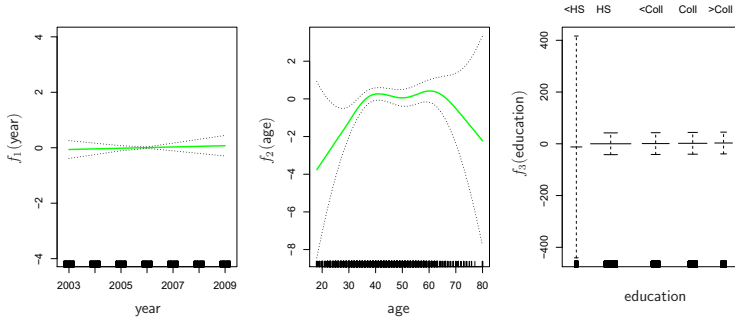
7.7.2 GAMs for Classification Problems

GAMs cũng dùng được khi Y qualitative. Để đơn giản, giả sử $Y \in \{0, 1\}$ và $p(X) = \Pr(Y = 1 | X)$. Logistic regression model (4.6) là

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p. \quad (7.17)$$

Left side là log odds của $P(Y = 1 | X)$ so với $P(Y = 0 | X)$; (7.17) biểu diễn quantity này như linear function của predictors. Extension tự nhiên cho non-linear relationships là

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p). \quad (7.18)$$



HÌNH 7.13. Với dataset *Wage*, logistic regression GAM (7.19) được fit cho binary response $I(\text{wage} > 250)$. Mỗi panel hiển thị fitted function và pointwise standard errors. Function thứ nhất linear theo *year*; function thứ hai là smoothing spline với năm degrees of freedom theo *age*; function thứ ba là step function cho *education*. Standard errors rất rộng ở level <HS.

Equation (7.18) là logistic regression GAM và có cùng pros/cons như case quantitative response. Ta fit GAM trên *Wage* để predict probability income vượt \$250,000 mỗi năm:

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + \beta_1 \times \text{year} + f_2(\text{age}) + f_3(\text{education}), \quad (7.19)$$

trong đó

$$p(X) = \Pr(\text{wage} > 250 | \text{year}, \text{age}, \text{education}).$$

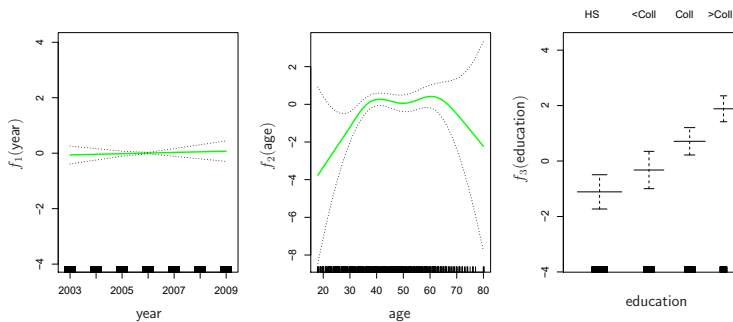
Một lần nữa f_2 dùng smoothing spline năm degrees of freedom, còn f_3 dùng step function qua dummy variables cho levels của education.

Fit thu được ở Figure 7.13. Last panel đáng ngờ vì confidence intervals rất rộng ở level <HS. Thực tế category này không có response bằng

one: không individual nào có less than high-school education kiếm hơn \$250,000/năm. Vì vậy ta refit GAM sau khi exclude category này. Result ở Figure 7.14. Như Figures 7.11 và 7.12, ba panels có vertical scales tương tự để so sánh relative contribution của variables. Ta thấy age và education có effect lớn hơn year lên probability trở thành high earner.

7.8 Lab: Non-Linear Modeling

Trong lab này, ta demonstrate một số nonlinear model đã bàn trong chapter. Dataset **Wage** được dùng xuyên suốt, và ta sẽ thấy nhiều complex non-linear fitting procedure có thể implement khá dễ trong Python.



HÌNH 7.14. Cùng model như Figure 7.13 được fit lại sau khi exclude observations có *education*=<HS. Bây giờ ta thấy education cao hơn có xu hướng associated với salary cao hơn.

Như thường lệ, ta bắt đầu với các standard imports.

In [1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
                          poly,
                          ModelSpec as MS)
from statsmodels.stats.anova import anova_lm
```

Ta cũng gom các import mới cần cho lab; nhiều item được phát triển riêng cho package ISLP.

In [2]:

```
from pygam import (s as s_gam,
                  l as l_gam,
                  f as f_gam,
                  LinearGAM,
                  LogisticGAM)
from ISLP.transforms import (BSpline,
                             NaturalSpline)
from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
                        degrees_of_freedom,
                        plot as plot_gam,
                        anova as anova_gam)
```

7.8.1 Polynomial Regression and Step Functions

Ta bắt đầu bằng cách reproduce Figure 7.1. Trước hết load data.

```
In [3]: Wage = load_data('Wage')
y = Wage['wage']
age = Wage['age']
```

Trong phần lớn lab, response là `Wage['wage']` đã lưu vào `y`. Như Section 3.6.6, ta dùng `poly()` để tạo model matrix fit fourth-degree polynomial theo `age`.

```
In [4]: poly_age = MS([poly('age', degree=4)]).fit(Wage)
M = sm.OLS(y, poly_age.transform(Wage)).fit()
summarize(M)
```

```
Out[4]:
```

	coef	std err	t	P> t
intercept	111.7036	0.729	153.283	0.000
poly(age, degree=4) [0]	447.0679	39.915	11.201	0.000
poly(age, degree=4) [1]	-478.3158	39.915	-11.983	0.000
poly(age, degree=4) [2]	125.5217	39.915	3.145	0.002
poly(age, degree=4) [3]	-77.9112	39.915	-1.952	0.051

Polynomial này được construct bằng helper function `poly()`, tạo transformer đặc biệt `Poly()` (theo terminology của `sklearn` cho feature transformations, như `PCA()` ở Section 6.5.3) để evaluate polynomial dễ dàng trên new data points. `poly()` thiết lập transformation; `Poly()` là workhorse thực sự compute transformation. Xem thêm discussion về transformations ở page 118.

transformer
helper

Trong code trên, first line chạy method `fit()` trên dataframe `Wage`. Việc này recompute và lưu dưới dạng attributes mọi parameter mà `Poly()` cần từ training data; các parameter này sau đó được dùng cho mọi evaluation của `transform()`. Chẳng hạn chúng được dùng ở second line và trong plotting function phía dưới.

Bây giờ tạo grid của age values để predict.

```
In [5]: age_grid = np.linspace(age.min(),
                             age.max(),
                             100)
age_df = pd.DataFrame({'age': age_grid})
```

Ta muốn plot data và thêm fourth-degree polynomial fit. Vì sẽ làm nhiều plot tương tự, ta viết một function tạo các ingredient và plot. Function nhận model specification (ở đây là basis do transform xác định) và grid age values; nó tạo fitted curve cùng 95% confidence bands. Argument `basis` giúp dùng lại function với nhiều transform khác, chẳng hạn splines.

```
In [6]: def plot_wage_fit(age_df,
                        basis,
                        title):
    X = basis.transform(Wage)
    Xnew = basis.transform(age_df)
    M = sm.OLS(y, X).fit()
    preds = M.get_prediction(Xnew)
    bands = preds.conf_int(alpha=0.05)
    fig, ax = subplots(figsize=(8,8))
```

```

ax.scatter(age,
           y,
           facecolor='gray',
           alpha=0.5)
for val, ls in zip([preds.predicted_mean,
                  bands[:,0],
                  bands[:,1]],
                  ['b', 'r--', 'r--']):
    ax.plot(age_df.values, val, ls, linewidth=3)
ax.set_title(title, fontsize=20)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
return ax

```

Argument `alpha` trong `ax.scatter()` thêm transparency cho points, giúp visualize density. Chú ý `zip()` trong for loop (Section 2.3.8): ta có ba line với color/line type khác nhau, và `zip()` bundle chúng thành iterators trong loop. ⁷

iterator

Ta plot fourth-degree polynomial fit bằng function vừa tạo.

In [7]:

```

plot_wage_fit(age_df,
              poly_age,
              'Degree-4 Polynomial');

```

Với polynomial regression ta phải chọn degree. Đôi khi ta đơn giản chọn degree 2 hoặc 3 để có nonlinear fit, nhưng có thể systematic hơn bằng hypothesis tests. Ta fit series model từ linear (degree 1) đến degree 5, rồi tìm simplest model đủ explain relationship giữa wage và age. Function `anova_lm()` thực hiện sequence ANOVA tests. Analysis of variance (ANOVA) kiểm định null hypothesis rằng model M_1 đủ explain data, so với alternative rằng cần model phức tạp hơn M_2 . Determination dựa trên F-test, và M_1, M_2 phải nested: predictor space của M_1 là subspace của predictor space của M_2 .

analysis of variance

In [8]:

```

models = [MS([poly('age', degree=d)])
          for d in range(1, 6)]
Xs = [model.fit_transform(Wage) for model in models]
anova_lm(*[sm.OLS(y, X_).fit()
           for X_ in Xs])

```

Out[8]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	2998.0	5.022e+06	0.0	NaN	NaN	NaN
1	2997.0	4.793e+06	1.0	228786.010	143.593	2.364e-32
2	2996.0	4.778e+06	1.0	15755.694	9.889	1.679e-03
3	2995.0	4.772e+06	1.0	6070.152	3.810	5.105e-02
4	2994.0	4.770e+06	1.0	1282.563	0.805	3.697e-01

Chú ý dấu `*` trong line `anova_lm()` ở trên. Function này nhận variable number of non-keyword arguments, ở đây là fitted models. Khi các model được cung cấp dưới dạng list, list phải prefix bằng `*`. P-value so sánh linear `models[0]` với quadratic `models[1]` gần như zero, cho thấy linear fit không đủ.⁸ Tương tự, p-value so sánh quadratic và cubic là 0.0017, nên quadratic cũng không đủ. P-value so sánh cubic và degree-four khoảng 5%,

⁷Theo cách nói của Python, “iterator” là object có finite number of values và có thể iterate, như trong loop.

⁸Index bắt đầu từ zero hơi dễ gây nhầm trong example polynomial degree, vì `models[1]` là quadratic chứ không phải linear.

còn degree-five không cần thiết vì p-value 0.37. Vì vậy cubic hoặc quartic polynomial đều là reasonable fit; lower-order hoặc higher-order model không được justify.

Thay vì `anova_lm()`, trong case này ta có thể lấy p-values ngắn gọn hơn nhờ `poly()` tạo orthogonal polynomials.

In [9]:

```
summarize(M)
```

Out[9]:

	coef	std err	t	P> t
intercept	111.7036	0.729	153.283	0.000
poly(age, degree=4) [0]	447.0679	39.915	11.201	0.000
poly(age, degree=4) [1]	-478.3158	39.915	-11.983	0.000
poly(age, degree=4) [2]	125.5217	39.915	3.145	0.002
poly(age, degree=4) [3]	-77.9112	39.915	-1.952	0.051

P-values giống nhau, và square của t-statistics bằng F-statistics từ `anova_lm()`; ví dụ:

In [10]:

```
(-11.983)**2
```

Out[10]:

```
143.59228
```

ANOVA method vẫn hoạt động dù ta không dùng orthogonal polynomials, miễn models nested. Ví dụ, ta compare ba model đều có linear term cho `education` và polynomial theo `age` với degree khác nhau:

In [11]:

```
models = [MS(['education', poly('age', degree=d)])
           for d in range(1, 4)]
XEs = [model.fit_transform(Wage)
        for model in models]
anova_lm([sm.OLS(y, X_).fit() for X_ in XEs])
```

Out[11]:

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	2997.0	3.902e+06	0.0	NaN	NaN	NaN
1	2996.0	3.759e+06	1.0	142862.701	113.992	3.838e-26
2	2995.0	3.754e+06	1.0	5926.207	4.729	2.974e-02

Một alternative cho hypothesis tests và ANOVA là chọn polynomial degree bằng cross-validation, như Chapter 5.

Tiếp theo ta predict liệu một individual có earning trên \$250,000 mỗi năm hay không. Cách làm gần như trước: tạo response vector phù hợp, rồi dùng `sm.GLM()` với binomial family để fit polynomial logistic regression model.

In [12]:

```
X = poly_age.transform(Wage)
high_earn = Wage['high_earn'] = y > 250 # shorthand
glm = sm.GLM(y > 250,
              X,
              family=sm.families.Binomial())
B = glm.fit()
summarize(B)
```

Out[12]:

	coef	std err	z	P> z
intercept	-4.3012	0.345	-12.457	0.000
poly(age, degree=4) [0]	71.9642	26.133	2.754	0.006
poly(age, degree=4) [1]	-85.7729	35.929	-2.387	0.017

```
poly(age, degree=4)[2]      34.1626   19.697   1.734   0.083
poly(age, degree=4)[3]     -47.4008   24.105  -1.966   0.049
```

Một lần nữa ta predict bằng method `get_prediction()`.

```
In [13]: newX = poly_age.transform(age_df)
         preds = B.get_prediction(newX)
         bands = preds.conf_int(alpha=0.05)
```

Bây giờ plot estimated relationship.

```
In [14]: fig, ax = subplots(figsize=(8,8))
         rng = np.random.default_rng(0)
         ax.scatter(age +
                     0.2 * rng.uniform(size=y.shape[0]),
                     np.where(high_earn, 0.198, 0.002),
                     fc='gray',
                     marker='|')
         for val, ls in zip([preds.predicted_mean,
                             bands[:,0],
                             bands[:,1]],
                             ['b', 'r--', 'r--']):
             ax.plot(age_df.values, val, ls, linewidth=3)
         ax.set_title('Degree-4 Polynomial', fontsize=20)
         ax.set_xlabel('Age', fontsize=20)
         ax.set_ylim([0,0.2])
         ax.set_ylabel('P(Wage > 250)', fontsize=20);
```

Ta vẽ age values của observations có wage trên 250 thành gray marks ở top, còn dưới 250 ở bottom. Một lượng nhỏ noise được thêm để jitter age values, tránh observations cùng age che nhau; plot kiểu này thường gọi là *rug plot*.

Để fit step function như Section 7.2, ta dùng `pd.qcut()` discretize age theo quantiles, rồi `pd.get_dummies()` tạo columns của model matrix cho categorical variable. Function này include mọi column của categorical thay vì drop một level như usual approach.

```
In [15]: cut_age = pd.qcut(age, 4)
         summarize(sm.OLS(y, pd.get_dummies(cut_age)).fit())
```

```
Out[15]:
```

	coef	std err	t	P> t
(17.999, 33.75]	94.1584	1.478	63.692	0.0
(33.75, 42.0]	116.6608	1.470	79.385	0.0
(42.0, 51.0]	119.1887	1.416	84.147	0.0
(51.0, 80.0]	116.5717	1.559	74.751	0.0

Ở đây `pd.qcut()` tự động chọn cutpoints theo quantiles 25%, 50%, 75%, tạo bốn regions. Ta cũng có thể specify quantiles trực tiếp thay vì argument 4. Với cuts không dựa trên quantiles, dùng `pd.cut()`. Cả `pd.qcut()` và `pd.cut()` trả về ordered categorical variable; regression model sau đó tạo dummy variables để dùng trong regression. Vì age là variable duy nhất trong model, \$94,158.40 là average salary của group dưới 33.75 tuổi, còn các coefficients khác là average salary ở các age group tương ứng. Prediction và plot có thể tạo tương tự polynomial fit.

rug plot

`pd.qcut()`
`pd.get_dummies()`

`pd.cut()`